Atividade n. 7

Efeito de aquisição por mosaico

i) Introdução

O sensor de uma câmera só mede a intensidade da luz, não a cor. Para conseguir uma imagem colorida com um sensor só, coloca-se em cima dele um filtro em mosaico, em que cada pixel deixa passar apenas uma cor (R, G ou B). O exemplo mais famoso é o filtro de Bayer (RG/GB), que usa mais pixels verdes porque nosso olho é mais sensível ao verde.

O problema é que cada pixel perde duas das três cores. Na prática, é como se cada canal fosse subamostrado. Essa intercalação gera efeitos indesejados: a imagem fica mais escura, aparece um padrão “xadrez” ou de listras e surgem cores falsas, principalmente em regiões brancas e cinzas. Esses efeitos são uma forma de aliasing e podem ser reduzidos com um filtro passa-baixas (anti-aliasing).

Uma alternativa é o sensor Foveon, que tem três camadas empilhadas e mede R, G e B no mesmo ponto, sem precisar de mosaico.

ii) Procedimento

Para cada imagem, o script faz o seguinte:

1. Cria máscaras binárias para R, G e B a partir do padrão (2×2 ou 3×3), repetidas até cobrir a imagem toda.
2. Multiplica cada canal da imagem pela sua máscara. Assim cada pixel fica com uma única cor, simulando o sensor. Essas são as Soluções I e II.
3. Na Solução III, aplica um filtro média 3×3 em cada canal e divide pela quantidade de amostras daquela cor na janela. Isso suaviza o padrão e recupera o brilho perdido.

Também medimos a fração de cada cor que sobra após o mosaico, a saturação nas regiões cinza/brancas (para quantificar a cor falsa) e a PSNR em relação à original.

iii) Resultados e comentários

3.1 Borboleta (alta saturação)



“Figura 1 – Borboleta: original (sup. esq.), mosaico 2×2 (sup. dir.), 3×3 (inf. esq.) e 3×3 com anti-aliasing (inf. dir.)”

Como quase toda a imagem é bem colorida (só 2,0% dos pixels são pouco saturados), a coloração artificial do 2×2 aparece pouco: o que mais se nota é a imagem escura e puxada para o verde. No 3×3 as cores ficam equilibradas, mas aparecem as listras. Com o anti-aliasing a imagem fica quase igual à original (PSNR de 31,2 dB, a maior das três), porque o fundo desfocado tem poucos detalhes finos para o filtro borrar.

3.2 Carro (baixa saturação)



“Figura 2 – Carro: original (sup. esq.), mosaico 2×2 (sup. dir.), 3×3 (inf. esq.) e 3×3 com anti-aliasing (inf. dir.)”

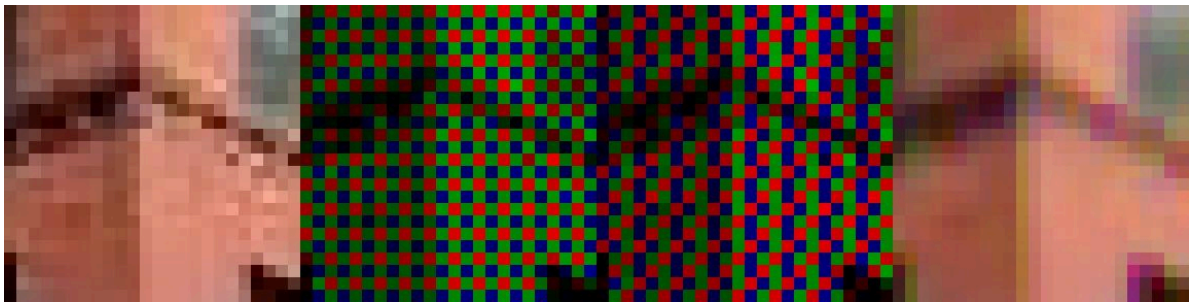
É a imagem onde o efeito de cor falsa fica mais evidente, já que 64% dela é cinza (céu e asfalto). No 2×2, essas áreas cinzas ficam completamente verdes. No 3×3 o cinza continua

cinza, só que mais escuro e listrado. Com o anti-aliasing o céu e o asfalto voltam a ficar neutros, e o carro amarelo mantém a cor certinha (PSNR de 29,8 dB).

3.3 Prédio (saturação mista)





“Figura 3 – Prédio: original (sup. esq.), mosaico 2×2 (sup. dir.), 3×3 (inf. esq.) e 3×3 com anti-aliasing (inf. dir.)”

Tem regiões saturadas (tijolo vermelho e céu azul) e regiões claras e cinzas (calçada e rua). O 2×2 deixa a calçada e a rua esverdeadas. No 3×3 as listras aparecem bastante por causa das muitas janelas e linhas verticais do prédio. O anti-aliasing resolve as listras, mas é onde a perda de nitidez mais aparece (PSNR de 22,9 dB, a menor), porque o filtro suaviza as bordas das janelas e deixa algumas franjas coloridas nelas.

3.4 Comparação

Nas tabelas a seguir, as colunas R, G e B indicam a fração de cada cor que foi mantida após o mosaico (1,00 = 100%). A coluna Sat. baixa mostra a saturação média nas regiões que eram cinzas ou brancas na imagem original: quanto maior o valor, mais cor falsa apareceu. A PSNR mede o quanto o resultado se parece com a original: quanto maior, mais parecido**.**

Tabela 1 - Borboleta (4288 x 2848 px)
Saturação média da original: 0.489 | pixels pouco saturados (S<0.15): 2.0%

Solução	R	G	B	Sat.baixa	PSNR(dB)
Original	1.00	1.00	1.00	0.117	-
I (2x2)	0.25	0.50	0.25	0.701	8.90
II (3x3)	0.33	0.33	0.33	0.155	8.83
III (3x3 + AA)	1.00	1.00	1.00	0.140	31.19

Tabela 2 - Carro (554 x 554 px)
Saturação média da original: 0.231 | pixels pouco saturados (S<0.15): 64.2%

Solução	R	G	B	Sat.baixa	PSNR(dB)
Original	1.00	1.00	1.00	0.065	-

I (2x2)	**0.25**	**0.50**	**0.25**	**0.684**	**8.26**
II (3x3)	**0.33**	**0.33**	**0.33**	**0.084**	**8.21**
III (3x3 + AA)	**1.00**	**1.00**	**1.00**	**0.075**	**29.82**

Tabela 3 - Prédio (387 x 516 px)
Saturação média da original: 0.414 | pixels pouco saturados (S<0.15): 13.0%

Solução	R	G	B	Sat.baixa	PSNR(dB)
Original	**1.00**	**1.00**	**1.00**	**0.098**	**-**
I (2x2)	**0.25**	**0.50**	**0.25**	**0.691**	**7.24**
II (3x3)	**0.33**	**0.33**	**0.33**	**0.192**	**7.32**
III (3x3 + AA)	**1.00**	**1.00**	**1.00**	**0.156**	**22.91**

Solução I (2×2): a imagem fica escura e bem esverdeada. Isso acontece porque o padrão tem o dobro de pixels verdes: sobram 50% do verde e só 25% do vermelho e do azul. Nas regiões brancas ou cinzas, o branco “vira” verde, e a saturação nessas áreas sobe de cerca de 0,1 para cerca de 0,7. No carro, que tem muito céu e asfalto cinza, a imagem inteira fica verde. Na borboleta o efeito incomoda menos, porque a imagem já é bem colorida.

Solução II (3×3): as três cores aparecem na mesma proporção (1/3 cada), então a coloração artificial cai bastante (no carro, a saturação nas áreas cinza fica em 0,08). A imagem continua escura e aparecem listras verticais. O motivo é que as colunas do padrão são diferentes entre si: uma é neutra, outra puxa para o verde e outra para o magenta, e essa alternância cria as listras.

Solução III (3×3 + anti-aliasing): o filtro passa-baixas “apaga” as listras e o xadrez, o brilho volta ao normal e as cores falsas praticamente somem. A PSNR sobe de cerca de 8 dB para valores entre 23 e 31 dB. Em troca, a imagem fica um pouco borrada e perde detalhes finos. Isso aparece mais no prédio, que tem muitas bordas, e menos na borboleta, que tem fundo desfocado.

iv) Conclusões

- Capturar cor por mosaico faz cada pixel perder informação. Sem tratamento, o resultado fica escuro, com padrões e cores falsas.
- A proporção de cores no padrão faz diferença: o 2×2 (mais verde) deixa a imagem esverdeada, e o 3×3 (cores equilibradas) evita isso, mas cria listras.

- As cores falsas aparecem mais em regiões pouco saturadas (branco e cinza), onde qualquer desequilíbrio entre R, G e B fica visível.
- O filtro anti-aliasing resolve a maior parte do problema, com o custo de um pouco de nitidez. Por isso, as câmeras reais usam algoritmos mais sofisticados para reconstruir as cores.

v) Comando e Códigos

>> lab7_mosaico

---

lab7_mosaico.m :

```
%% ======================================================================
%  IESB - Engenharia da Computacao
%  Processamento de Sinais Multimidia - Atividade de Laboratorio n. 7
%  Efeito de aquisicao por mosaico
%
%  Solucao I  : mosaico com padrao 2x2  [R G; G B]
%  Solucao II : mosaico com padrao 3x3  [R G B; B G R; G R B]
%  Solucao III: mosaico 3x3 + filtro anti-aliasing (passa-baixas 3x3)
%
%  Arquivos necessarios na mesma pasta:
%    gera_mascaras.m, aplica_mosaico.m, antialiasing_mosaico.m, sat_regiao.m
%    img1.jpg, img2.jpg, img3.jpg
%% ======================================================================
clear; close all; clc;

%% ---------------- Parametros ----------------------------------------
arquivos = {'img1.jpg', 'img2.jpg', 'img3.jpg'};
nomes    = {'Borboleta', 'Carro', 'Predio'};

padrao2x2 = ['RG';
             'GB'];

padrao3x3 = ['RGB';
             'BGR';
             'GRB'];

mostrarFig  = true;   % false = apenas salva as imagens, sem abrir janelas
```

```
tamFiltro   = 3;      % janela do anti-aliasing (3x3)
limiarSat   = 0.15;   % saturacao abaixo disso = regiao "pouco colorida"
tamZoom     = 24;     % lado (em pixels) do recorte ampliado
fatorZoom   = 12;     % ampliacao do recorte (vizinho mais proximo)

pastaSaida = 'resultados';
if ~exist(pastaSaida, 'dir'), mkdir(pastaSaida); end

%% ---------------- Processamento de cada imagem ----------------------
for k = 1:numel(arquivos)

  % --- Leitura e conversao para double em [0,1]
  img = imread(arquivos{k});
  if size(img, 3) == 1                      % se vier em tons de cinza
    img = repmat(img, [1 1 3]);
  end
  img = double(img) / double(intmax(class(img)));

  % --- Solucao I: mosaico 2x2
  sol1 = aplica_mosaico(img, padrao2x2);

  % --- Solucao II: mosaico 3x3
  [sol2, mR, mG, mB] = aplica_mosaico(img, padrao3x3);

  % --- Solucao III: mosaico 3x3 + anti-aliasing
  sol3 = antialiasing_mosaico(sol2, mR, mG, mB, tamFiltro);

  % --- Exibicao (mesmo formato do roteiro)
  if mostrarFig
  figure('Name', sprintf('Imagem %d - %s', k, nomes{k}), 'NumberTitle', 'off');
  subplot(2,2,1); imshow(img);  title('Original');
  subplot(2,2,2); imshow(sol1); title('Solucao I: mosaico 2x2');
  subplot(2,2,3); imshow(sol2); title('Solucao II: mosaico 3x3');
  subplot(2,2,4); imshow(sol3); title('Solucao III: 3x3 + anti-aliasing');
  end

  % --- Recorte ampliado (zoom) para enxergar o padrao pixel a pixel
  [M, N, ~] = size(img);
  li = round(M/2) - tamZoom/2 + (1:tamZoom);
  co = round(N/2) - tamZoom/2 + (1:tamZoom);
  ampl = @(x) x(kron(li, ones(1,fatorZoom)), kron(co, ones(1,fatorZoom)), :);
```

```
  if mostrarFig
  figure('Name', sprintf('Zoom %d - %s', k, nomes{k}), 'NumberTitle', 'off');
  subplot(1,4,1); imshow(ampl(img));  title('Original');
  subplot(1,4,2); imshow(ampl(sol1)); title('2x2');
  subplot(1,4,3); imshow(ampl(sol2)); title('3x3');
  subplot(1,4,4); imshow(ampl(sol3)); title('3x3 + AA');
  end

  % --- Salva os resultados
  base = fullfile(pastaSaida, sprintf('img%d', k));
  imwrite(img,  [base '_0_original.png']);
  imwrite(sol1, [base '_1_mosaico2x2.png']);
  imwrite(sol2, [base '_2_mosaico3x3.png']);
  imwrite(sol3, [base '_3_mosaico3x3_AA.png']);
  imwrite([ampl(img) ampl(sol1) ampl(sol2) ampl(sol3)], [base '_4_zoom.png']);

  %% ------------- Medidas para embasar os comentarios ---------------
  % 1) Fracao do brilho de cada canal que "sobrevive" ao mosaico
  frac = @(x) squeeze(sum(sum(x, 1), 2))' ./ squeeze(sum(sum(img, 1), 2))';
  f1 = frac(sol1);  f2 = frac(sol2);  f3 = frac(sol3);

  % 2) Coloracao artificial: saturacao media nas regioes originalmente
  %    pouco saturadas (brancos, cinzas). Calculada sobre a imagem
  %    suavizada 3x3 (o que o olho percebe a distancia, sem o "xadrez").
  hsvO  = rgb2hsv(img);
  baixa = hsvO(:,:,2) < limiarSat;
  suav  = @(x) cat(3, conv2(x(:,:,1), ones(3)/9, 'same'), ...
                      conv2(x(:,:,2), ones(3)/9, 'same'), ...
                      conv2(x(:,:,3), ones(3)/9, 'same'));
  satMedia = @(x) mean(mean(sat_regiao(suav(x), baixa)));

  % 3) PSNR em relacao a original
  psnr_db = @(x) 10*log10(1 / mean((x(:) - img(:)).^2));

  fprintf('\n=== Imagem %d (%s) - %dx%d px ===\n', k, nomes{k}, N, M);
  fprintf('Saturacao media da original: %.3f | pixels pouco saturados (S<%.2f): %.1f%%\n', ...
          mean(mean(hsvO(:,:,2))), limiarSat, 100*mean(baixa(:)));
  fprintf('%-22s %6s %6s %6s %12s %10s\n', 'Solucao', 'R', 'G', 'B', 'Sat.baixa', 'PSNR(dB)');
  fprintf('%-22s %6s %6s %6s %12.3f %10s\n', 'Original', '1.00', '1.00', '1.00', ...
          mean(sat_regiao(suav(img), baixa)), '-');
   fprintf('%-22s %6.2f %6.2f %6.2f %12.3f %10.2f\n', 'I    (2x2)',       f1, satMedia(sol1),
psnr_db(sol1));
```

```
   fprintf('%-22s %6.2f %6.2f %6.2f %12.3f %10.2f\n', 'II   (3x3)',       f2, satMedia(sol2),
psnr_db(sol2));
   fprintf('%-22s %6.2f %6.2f %6.2f %12.3f %10.2f\n', 'III  (3x3 + AA)', f3, satMedia(sol3),
psnr_db(sol3));
end

fprintf('\nImagens salvas na pasta "%s".\n', pastaSaida);
```

---

gera_mascaras.m :

```
function [mR, mG, mB] = gera_mascaras(padrao, M, N)
% GERA_MASCARAS  Gera as mascaras binarias R, G e B de um filtro em mosaico.
%
%   [mR, mG, mB] = gera_mascaras(padrao, M, N)
%
%   padrao : matriz de caracteres com o padrao basico (ex.: ['RG';'GB'])
%   M, N   : dimensoes (linhas, colunas) da imagem
%
%   Cada mascara vale 1 nas posicoes (i,j) em que o sensor registra aquela
%   componente de cor e 0 nas demais. O padrao basico e repetido (tiling)
%   ate cobrir toda a imagem.

  [p, q] = size(padrao);

  % Quantas vezes o padrao precisa ser repetido em cada direcao
  nRep_lin = ceil(M / p);
  nRep_col = ceil(N / q);

  % Mascaras do bloco basico
  bR = double(padrao == 'R');
  bG = double(padrao == 'G');
  bB = double(padrao == 'B');

  % Repete o bloco e recorta no tamanho exato da imagem
  mR = repmat(bR, nRep_lin, nRep_col);  mR = mR(1:M, 1:N);
  mG = repmat(bG, nRep_lin, nRep_col);  mG = mG(1:M, 1:N);
  mB = repmat(bB, nRep_lin, nRep_col);  mB = mB(1:M, 1:N);
end
```

---

aplica_mosaico.m :

```
function [mosaico, mR, mG, mB] = aplica_mosaico(img, padrao)
% APLICA_MOSAICO  Simula a aquisicao de uma imagem colorida por mosaico.
%
%   [mosaico, mR, mG, mB] = aplica_mosaico(img, padrao)
%
%   img    : imagem RGB em double, valores em [0, 1]
%   padrao : padrao basico do mosaico (ex.: ['RG';'GB'] ou ['RGB';'BGR';'GRB'])
%
%   Em cada pixel apenas UMA componente (R, G ou B) e mantida, como num
%   sensor com filtro de cores em mosaico; as outras duas sao zeradas.
%   O resultado e exibido como imagem RGB (o "RAW" visto em cores).

  [M, N, ~] = size(img);
  [mR, mG, mB] = gera_mascaras(padrao, M, N);

  mosaico = zeros(M, N, 3);
  mosaico(:,:,1) = img(:,:,1) .* mR;   % so passa vermelho onde o filtro e R
  mosaico(:,:,2) = img(:,:,2) .* mG;   % so passa verde    onde o filtro e G
  mosaico(:,:,3) = img(:,:,3) .* mB;   % so passa azul     onde o filtro e B
end
```

---

antialiasing_mosaico.m :

```
function saida = antialiasing_mosaico(mosaico, mR, mG, mB, tam)
% ANTIALIASING_MOSAICO  Filtro passa-baixas (anti-aliasing) aplicado ao mosaico.
%
%   saida = antialiasing_mosaico(mosaico, mR, mG, mB, tam)
%
%   mosaico    : imagem em mosaico (saida de aplica_mosaico)
%   mR,mG,mB   : mascaras usadas na aquisicao
%   tam        : tamanho da janela do filtro media (ex.: 3 -> 3x3)
%
%   Para cada canal, calcula a MEDIA dos pixels daquele canal que existem
%   dentro de uma janela tam x tam (convolucao normalizada):
%
%       canal_filtrado = conv2(canal .* mascara, h) ./ conv2(mascara, h)
%
%   O numerador soma (com peso) os valores amostrados; o denominador conta
%   quantas amostras daquele canal cairam na janela. Assim:
%     - o filtro passa-baixas remove a oscilacao de alta frequencia
%       (o "xadrez" / listras) gerada pela intercalacao das cores;
```

```
%     - a divisao recupera o brilho original (cada canal so ocupa
%       uma fracao dos pixels do sensor).

  h = ones(tam) / tam^2;               % filtro media (passa-baixas) tam x tam
  mascaras = {mR, mG, mB};
  saida = zeros(size(mosaico));

  for c = 1:3
    num = conv2(mosaico(:,:,c), h, 'same');
    den = conv2(mascaras{c},    h, 'same');
    den(den == 0) = eps;               % evita divisao por zero nas bordas
    saida(:,:,c) = num ./ den;
  end

  saida = min(max(saida, 0), 1);       % garante valores em [0, 1]
end
```

---

sat_regiao.m :

```
function s = sat_regiao(x, mascara)
% SAT_REGIAO  Saturacao (canal S do HSV) dos pixels de x indicados pela mascara.
  h = rgb2hsv(min(max(x, 0), 1));
  S = h(:,:,2);
  s = S(mascara);
end
```